

TAR Command in Linux

What is **tar**?

tar stands for **Tape Archive**.

The **tar** command is used in Linux/Unix systems to:

- Combine multiple files and directories into a single archive file
- Compress archive files
- Extract archive files

Originally, **tar** was used to store files on **magnetic tapes**, but today it is widely used for:

-  System backups
 -  Software distribution
 -  File compression
 -  Data transfer
-

Common Archive File Extensions

Extension	Meaning
.tar	Archive only (no compression)
.tar.gz	Archive + gzip compression
.tar.bz2	Archive + bzip2 compression
.tar.xz	Archive + xz compression

Basic Syntax of tar

`tar [options] [archive-file] [files or directories]`

◆ Example:

`tar -cvf backup.tar file1 file2`

◆ Meaning of Options Used:

Option	Meaning
<code>c</code>	Create archive
<code>v</code>	Verbose (show files being processed)
<code>f</code>	Specify archive filename

Important **tar** Options

Option	Meaning
<code>-c</code>	Create archive
<code>-x</code>	Extract archive
<code>-v</code>	Verbose output
<code>-f</code>	Specify archive filename
<code>-t</code>	List archive contents
<code>-z</code>	Compress using gzip
<code>-j</code>	Compress using bzip2
<code>-J</code>	Compress using xz
<code>-C</code>	Extract to specific directory
<code>--exclude</code>	Exclude specific files

Creating an Archive

♦ Create a Simple Archive

```
tar -cvf backup.tar file1.txt file2.txt
```

♦ Output Example:

```
file1.txt  
file2.txt
```

♦ Archive a Directory

```
tar -cvf project.tar project/
```

This creates **project.tar** containing the entire **project** directory.

Extracting an Archive

♦ Extract a .tar File

```
tar -xvf backup.tar
```

This extracts all files in the current directory.

♦ Extract to a Specific Directory

```
tar -xvf backup.tar -C /home/user/restore/
```

Files will be extracted into the **restore** directory.

Listing Archive Contents

To view files inside an archive without extracting:

```
tar -tvf backup.tar
```

♦ **Example Output:**

```
-rw-r--r-- user user 120 Mar 3 file1.txt  
-rw-r--r-- user user 150 Mar 3 file2.txt
```

Creating Compressed Archives

♦ **gzip Compression**

```
tar -czvf backup.tar.gz folder/
```

z → Enables gzip compression

♦ **bzip2 Compression**

```
tar -cjvf backup.tar.bz2 folder/
```

j → Enables bzip2 compression

♦ **xz Compression**

```
tar -cJvf backup.tar.xz folder/
```

J → Enables xz compression

Extracting Compressed Archives

♦ **Extract .tar.gz**

```
tar -xzf backup.tar.gz
```

♦ **Extract .tar.bz2**

```
tar -xjf backup.tar.bz2
```

◆ Extract .tar.xz

```
tar -xJvf backup.tar.xz
```

🎯 Extract Specific File from Archive

```
tar -xvf backup.tar file1.txt
```

Only **file1.txt** will be extracted.

🚫 Exclude Files While Creating Archive

```
tar --exclude="*.log" -cvf backup.tar folder/
```

This excludes all **.log** files from the archive.

+ Append Files to Existing Archive

```
tar -rvf backup.tar newfile.txt
```

r → Append files to an existing archive

✅ Verify Archive

```
tar -tvf backup.tar
```

Used to verify the contents of the archive.

🌟 Advantages of **tar**

- ✓ Combines multiple files into one archive
- ✓ Useful for system backups
- ✓ Supports multiple compression methods

- ✓ Preserves file permissions and ownership
 - ✓ Efficient for file transfer
-

Quick Command Summary

Task	Command
Create archive	<code>tar -cvf file.tar folder/</code>
Extract archive	<code>tar -xvf file.tar</code>
List contents	<code>tar -tvf file.tar</code>
Create gzip archive	<code>tar -czvf file.tar.gz folder/</code>
Extract gzip archive	<code>tar -xzvf file.tar.gz</code>
Create bzip2 archive	<code>tar -cjvf file.tar.bz2 folder/</code>
Extract bzip2 archive	<code>tar -xjvf file.tar.bz2</code>

ZIP and UNZIP Commands in Linux

◆ Introduction to **zip**

- **zip** is used to:
 - Compress files
 - Combine multiple files into a single compressed file
- It reduces file size for:
 - Easy transfer

- Email attachments
 - Storage saving
 - Commonly used in Windows and Linux systems
 - Creates files with `.zip` extension
-

◆ Basic Syntax of `zip`

General Syntax:

`zip [options] archive_name.zip files`

Example:

`zip backup.zip file1.txt file2.txt`

- Creates `backup.zip`
 - Compresses `file1.txt` and `file2.txt`
-

◆ Important `zip` Options ⚙️

- `-r` → Compress directory recursively
- `-v` → Verbose output
- `-u` → Update existing zip file
- `-d` → Delete file from zip archive
- `-e` → Encrypt zip file (password protection)
- `-x` → Exclude files

◆ Creating ZIP Archives

✓ A) Zip Single File

zip file.zip file1.txt

✓ B) Zip Multiple Files

zip backup.zip file1.txt file2.txt

✓ C) Zip a Directory (Important)

zip -r project.zip project/

- **-r** is mandatory for directories
 - Compresses entire folder with subfolders
-

◆ 🔒 Create Password Protected ZIP

zip -e secure.zip file1.txt

- Prompts for password
 - Encrypts the archive
-

◆ 🚫 Exclude Files While Zipping

zip -r backup.zip folder/ -x "*.log"

- Excludes `.log` files
-

◆ Introduction to unzip

- `unzip` is used to:
 - Extract `.zip` files
 - View contents
 - Extract to specific directory
 - It works with `.zip` archives only
-

◆ Basic Syntax of `unzip`

`unzip archive_name.zip`

Example:

`unzip backup.zip`

- Extracts files in current directory
-

◆ Extract ZIP to Specific Directory

`unzip backup.zip -d /home/user/restore/`

- `-d` specifies destination directory
-

◆ 📄 List ZIP Contents Without Extracting

`unzip -l backup.zip`

- Displays list of files inside archive
 - Shows file size and date
-

◆ 🔄 Overwrite Files While Extracting

`unzip -o backup.zip`

- `-o` → Overwrites existing files without asking
-

◆ ❌ Extract Specific File from ZIP

`unzip backup.zip file1.txt`

- Extracts only `file1.txt`
-

◆ 🗑️ Delete File from ZIP Archive

`zip -d backup.zip file1.txt`

- Removes `file1.txt` from archive
-

◆ ☀️ Advantages of ZIP

- Widely supported (Windows, Linux, Mac)

- Easy to use
 - Supports password protection
 - Good compression ratio
 - Suitable for sharing files
-

◆ Quick Command Summary

- Create zip → `zip backup.zip file1.txt`
 - Zip directory → `zip -r project.zip project/`
 - Password protect → `zip -e secure.zip file1.txt`
 - Extract zip → `unzip backup.zip`
 - Extract to directory → `unzip backup.zip -d /path/`
 - List contents → `unzip -l backup.zip`
 - Delete file from zip → `zip -d backup.zip file1.txt`
-

Which One Should You Use?

Use TAR When:

- You are working on Linux servers.
- You are taking system backups.
- You need to preserve:

- File permissions
 - Ownership
 - Symbolic links
 - You want better compression (using `.tar.gz` or `.tar.xz`).
 - You are doing DevOps/server administration tasks.
-

✓ Use ZIP When:

- You want to share files with Windows users.
 - You are sending files via email.
 - You need password protection.
 - You want simple compression.
-

3 STAT Command in Linux

◆ Introduction to `stat`

- `stat` stands for **status**.
- It is used to display **detailed information** about a file or directory.
- Shows:
 - File size
 - Permissions
 - Owner & Group

- Inode number
 - Access, Modify & Change time
 - Very useful for:
 - Troubleshooting
 - Permission checking
 - File auditing
 - DevOps & system administration
-

◆ Basic Syntax

stat [options] filename

 **Example:**

stat file1.txt

◆ Example Output

File: file1.txt

Size: 120 Blocks: 8 IO Block: 4096 regular file

Device: 802h/2050d Inode: 1234567 Links: 1

Access: (0644/-rw-r--r--) Uid: (1000/user) Gid: (1000/user)

Access: 2026-03-04 10:20:15

Modify: 2026-03-04 09:30:10

Change: 2026-03-04 09:30:10

◆ Understanding Important Fields

File

- Name of the file.

Size

- File size in bytes.

Inode

- Unique identification number of the file in filesystem.

Links

- Number of hard links.

Access (0644/-rw-r--r--)

- Numeric permission → 0644
- Symbolic permission → -rw-r--r--

Uid / Gid

- Owner ID
- Group ID

Access

- Last time file was opened/read (atime)

Modify

- Last time file content was changed (**mtime**)

⚙️ Change

- Last time file metadata changed (**ctime**)

◆ Check Directory Details

stat folder_name

Shows:

- Directory permissions
- Directory size
- Timestamps

4 FIND Command in Linux

◆ Introduction to **find**

- **find** is used to search for:
 - Files
 - Directories
- It searches inside a specified directory path.
- It can search based on:

- Name
 - Size
 - Type
 - Permission
 - Owner
 - Time (modified, accessed)
 - Very powerful command used in:
 - System administration
 - Log file management
 - Cleanup operations
 - Automation scripts
-

◆ Basic Syntax of **find**

`find [path] [options] [expression]`

Example:

`find /home -name file1.txt`

- Searches for `file1.txt` inside `/home`
-

◆ Search by Name

Find a file by exact name

`find . -name "file1.txt"`

✓ Case-insensitive search

```
find . -iname "file1.txt"
```

◆ Search by File Type

- **f** → Regular file
- **d** → Directory
- **l** → Symbolic link

Example:

Find only files:

```
find . -type f
```

Find only directories:

```
find . -type d
```

◆ Search by Extension

```
find . -name "*.log"
```

Finds all **.log** files.

◆ Search by Size

- **k** → Kilobytes
- **M** → Megabytes
- **G** → Gigabytes

Example:

Find files greater than 10MB:

```
find . -size +10M
```

Find files less than 1MB:

```
find . -size -1M
```

◆ Search by Permissions

Find files with permission 777:

```
find . -perm 777
```

Find files with at least 777:

```
find . -perm -777
```

◆ Search by Owner

```
find . -user username
```

Finds files owned by a specific user.

◆ Search by Time

- `-mtime` → Modified time (days)
- `-atime` → Access time
- `-ctime` → Change time

Examples:

Find files modified in last 1 day:

```
find . -mtime -1
```

Find files older than 7 days:

```
find . -mtime +7
```

◆ Perform Action on Found Files (Very Important)

A) Delete files older than 7 days

```
find . -mtime +7 -delete
```

B) Execute a command on found files

```
find . -name "*.log" -exec rm {} \;
```

- `{}` → Represents found file
 - `\;` → Ends command
-

◆ Find and Move Files

```
find . -name "*.txt" -exec mv {} /home/user/docs/ \;
```

Moves all `.txt` files to docs folder.

☀ Advantages of `find`

- Very powerful search tool
- Supports multiple conditions
- Can execute commands automatically

- Useful for server maintenance
 - Used in automation scripts
-

Quick Command Summary

- Find file by name → `find . -name "file.txt"`
 - Find directory → `find . -type d`
 - Find by size → `find . -size +10M`
 - Find by permission → `find . -perm 777`
 - Find by owner → `find . -user username`
 - Delete old files → `find . -mtime +7 -delete`
 - Execute command → `find . -exec command {} \;`
-

5 CUT Command in Linux

◆ Introduction to **cut**

- **cut** is used to **extract specific parts of text** from a file or command output.
- It works line by line.
- Commonly used for:
 - Extracting columns from files

- Processing CSV files
 - Log analysis
 - Shell scripting
 - It can cut based on:
 - Character position
 - Byte position
 - Field (column)
-

◆ Basic Syntax

cut [option] filename

Or with pipe:

command | cut [option]

◆ Important Options ⚙

- **-c** → Cut by character position
 - **-b** → Cut by byte position
 - **-d** → Specify delimiter
 - **-f** → Select field (column)
 - **--complement** → Print everything except selected part
-

◆ Cut by Character Position ✂

Example File (data.txt)

Ashvini

Rahul

Priya

✓ Extract first 3 characters

```
cut -c 1-3 data.txt
```

Output:

Ash

Rah

Pri

✓ Extract specific characters

```
cut -c 2,4 data.txt
```

- Extracts 2nd and 4th character.
-

◆ Cut by Field (Column) 📊

Most useful feature of `cut`.

Example File (employees.txt)

101:Ashvini:IT:50000

102:Rahul:HR:40000

103:Priya:Admin:45000

✓ Extract 1st Field

```
cut -d ":" -f 1 employees.txt
```

Output:

101

102

103

✓ Extract Name Field

```
cut -d ":" -f 2 employees.txt
```

Output:

Ashvini

Rahul

Priya

Explanation:

- `-d ":"` → Delimiter is colon
 - `-f 2` → Select 2nd field
-

✓ Extract Multiple Fields

```
cut -d ":" -f 1,3 employees.txt
```

Output:

101:IT

102:HR

103:Admin

◆ Cut from Command Output (Very Important)

Example: Extract username from `/etc/passwd`

```
cut -d ":" -f 1 /etc/passwd
```

- Displays only usernames.
-

Example: Extract specific column from `ls -l`

```
ls -l | cut -c 1-10
```

- Displays permission column.
-

◆ Use Complement Option

Print everything except selected field:

```
cut -d ":" -f 2 --complement employees.txt
```

- Prints all fields except 2nd field.
-

★ Advantages of `cut`

- Simple and fast
- Useful for column extraction

- Works well with pipes
 - Important for shell scripting
 - Lightweight compared to awk
-

6 SORT Command in Linux

◆ Introduction to **sort**

- **sort** is used to **arrange lines of text in a specific order**.
 - It works line by line.
 - Default sorting:
 - Alphabetical (A → Z)
 - Commonly used for:
 - Sorting names
 - Sorting numbers
 - Log analysis
 - Data processing
 - Removing duplicates (with **-u**)
-

◆ Basic Syntax

`sort [options] filename`

Or with pipe:

command | sort

◆ Basic Example

📌 **File: names.txt**

Rahul

Ashvini

Priya

Karan

✅ **Command:**

sort names.txt

Output:

Ashvini

Karan

Priya

Rahul

(Default → Alphabetical order)

◆ Important **sort** Options ⚙️

Option	Meaning
--------	---------

-r	Reverse order
-n	Numeric sort
-k	Sort by column
-t	Specify delimiter
-u	Remove duplicates
-f	Ignore case
-o	Save output to file

◆ Reverse Sorting

`sort -r names.txt`

Output:

Rahul

Priya

Karan

Ashvini

◆ Numeric Sorting

 **File: numbers.txt**

10

2

50

1

 **Without -n**

sort numbers.txt

Output:

1

10

2

50

(Alphabetical sorting)

 **With -n**

sort -n numbers.txt

Output:

1

2

10

50

 -n is very important for numbers.

◆ Sort by Column (Field)

 **File: employees.txt**

101 Ashvini 50000

102 Rahul 40000

103 Priya 45000

✓ **Sort by Salary (3rd Column)**

`sort -k 3 -n employees.txt`

Output:

102 Rahul 40000

103 Priya 45000

101 Ashvini 50000

◆ Sort with Delimiter

If fields are separated by colon:

101:Ashvini:50000

102:Rahul:40000

`sort -t ":" -k 3 -n employees.txt`

- `-t ":"` → Sets delimiter
 - `-k 3` → Sort by 3rd field
-

◆ Remove Duplicate Lines

 **File:**

IT

HR

Admin

IT

HR

`sort -u dept.txt`

Output:

Admin

HR

IT

 `-u` = unique values

◆ Ignore Case Sorting

`sort -f names.txt`

- Ignores uppercase/lowercase differences.
-

◆ Save Sorted Output to File

`sort names.txt -o sorted_names.txt`

◆ Combine sort with Other Commands 🔥

✓ Sort usernames from /etc/passwd

```
cut -d ":" -f 1 /etc/passwd | sort
```

✓ Sort and remove duplicates

```
sort file.txt | uniq
```

OR

```
sort -u file.txt
```

🔍 Difference Between Alphabetical & Numeric Sorting

Without **-n**

With **-n**

Alphabetical

Numeric

1, 10, 2

1, 2, 10

★ Advantages of **sort**

- Organizes data easily
- Works well with pipes
- Important for log processing
- Removes duplicates

- Used in scripting & automation

🔥 Difference Between **find**, **cut**, and **sort**

Feature	find	cut	sort
Purpose	Search files/directories	Extract columns	Arrange data
Works On	File system	Text fields	Lines of text
Based On	File name, size, time, type	Character/field position	Alphabetical/Numerical order
Output	File paths	Selected columns	Sorted lines
Used For	File management	Data extraction	Data organization

7 | (Pipe) Command in Linux

The **pipe (|)** is used to connect multiple commands.

👉 It sends the **output of one command** as **input to another command**.

◆ Basic Syntax

`command1 | command2`

✓ Output of `command1` → Input of `command2`

◆ Why Pipe is Important?

- Combine multiple commands
 - Filter data step by step
 - Perform complex operations easily
 - Used heavily in automation & scripting
-

◆ Simple Example

`ls | sort`

✓ `ls` → lists files

✓ `sort` → sorts them alphabetically

◆ Practical Examples

1 Find Word in File

`cat data.txt | grep "Rahul"`

✓ `cat` gives content

✓ `grep` filters Rahul

(⚠ Better way is: `grep "Rahul" data.txt` — but this is for understanding pipe)

2 Count Number of Files

`ls | wc -l`

✓ `ls` → list files

✓ `wc -l` → count lines

3 Extract Column and Sort

`cat data.txt | awk '{print $2}' | sort`

✓ Extract 2nd column

✓ Sort the names

4 Find Largest File

`ls -l | sort -k5 -n`

✓ Sort by file size column

5 Search Process

`ps -ef | grep nginx`

✓ Shows running nginx process

♦ Multiple Pipes

You can chain many commands:

`command1 | command2 | command3 | command4`

Example:

```
cat data.txt | grep "Rahul" | awk '{print $3}' | sort -n
```

- ✓ Filter Rahul
 - ✓ Get salary
 - ✓ Sort salary
-

8 GREP Command in Linux

◆ Introduction to **grep**

- **grep** stands for **Global Regular Expression Print**.
 - It is used to:
 - Search text inside files
 - Filter command output
 - Find matching patterns
 - One of the most powerful text-processing commands in Linux.
 - Commonly used in:
 - Log analysis
 - Searching configuration files
 - Troubleshooting
 - DevOps automation
-

◆ Basic Syntax

grep [options] pattern filename

Or with pipe:

command | grep pattern

◆ Basic Example

 **File: data.txt**

Ashvini

Rahul

Priya

Rahul

 **Search for "Rahul"**

grep "Rahul" data.txt

Output:

Rahul

Rahul

◆ Important **grep** Options ⚙️

Option	Meaning
-i	Ignore case
-v	Invert match (exclude pattern)

<code>-n</code>	Show line number
<code>-c</code>	Count matching lines
<code>-l</code>	Show only file names
<code>-r</code>	Recursive search
<code>-w</code>	Match whole word
<code>-A</code>	Show lines After match
<code>-B</code>	Show lines Before match
<code>-C</code>	Show lines Before & After

◆ Ignore Case Search

```
grep -i "rahul" data.txt
```

Matches:

- Rahul
- RAHUL
- rahul

◆ Show Line Numbers

```
grep -n "Rahul" data.txt
```

Output:

```
2:Rahul
```

```
4:Rahul
```

◆ Count Matching Lines

```
grep -c "Rahul" data.txt
```

Output:

```
2
```

◆ Invert Match (Exclude Pattern)

```
grep -v "Rahul" data.txt
```

Output:

```
Ashvini
```

```
Priya
```

◆ Search Recursively in Directory

```
grep -r "error" /var/log/
```

Searches for "error" inside all files under `/var/log`.

◆ Match Whole Word

```
grep -w "IT" employees.txt
```

Matches only exact word "IT", not "ITDept".

◆ Show Lines Before & After Match

Show 2 lines after match

```
grep -A 2 "error" logfile.txt
```

Show 2 lines before match

```
grep -B 2 "error" logfile.txt
```

Show 2 lines before and after

```
grep -C 2 "error" logfile.txt
```

◆ Use grep with Other Commands 🔥

✓ Find running process

```
ps -ef | grep nginx
```

✓ Find logged-in user

```
who | grep Ashvini
```

✓ Search usernames

```
cut -d ":" -f 1 /etc/passwd | grep root
```

◆ Regular Expression Example

Find lines starting with "A"

```
grep "^A" data.txt
```

Find lines ending with "I"

```
grep "I$" data.txt
```

☀ Advantages of **grep**

- Very fast search tool
 - Supports regular expressions
 - Works with pipes
 - Essential for log analysis
 - Powerful for troubleshooting
-

9 AWK Command

awk is a powerful text processing command used for:

- Pattern scanning
- Data extraction
- Report generation
- Field-based processing

It is mainly used for structured text like logs, CSV files, etc.

◆ Basic Syntax

`awk 'pattern { action }' filename`

If pattern is true → action runs.

◆ Default Behavior of awk

- Reads file line by line
 - Splits each line into fields
 - Default field separator = space
 - Fields are stored as:
 - `$1` → First column
 - `$2` → Second column
 - `$3` → Third column
 - `$0` → Entire line
 - `NR` → Line number
 - `NF` → Number of fields
-

◆ Example File (data.txt)

1 Ashvini 5000

2 Rahul 6000

3 Sneha 7000

◆ Common awk Examples

1 Print Entire File

```
awk '{print}' data.txt
```

2 Print Specific Column

```
awk '{print $2}' data.txt
```

Output:

Ashvini

Rahul

Sneha

3 Print Multiple Columns

```
awk '{print $1, $3}' data.txt
```

4 Print Line Number

```
awk '{print NR, $0}' data.txt
```

5 Condition Based Filtering

Print salary greater than 6000:

```
awk '$3 > 6000' data.txt
```

6 Change Field Separator (-F)

If file is comma separated:

```
1,Ashvini,5000
```

Command:

```
awk -F "," '{print $2}' data.txt
```

7 BEGIN and END Block

```
awk 'BEGIN {print "Report Start"} {print $2} END {print "Report End"}' data.txt
```

8 Sum of a Column

```
awk '{sum += $3} END {print sum}' data.txt
```

9 Print Last Column

```
awk '{print $NF}' data.txt
```

◆ Built-in Variables

Variable	Meaning
<code>\$0</code>	Entire line

<code>\$1, \$2 . .</code>	Fields
<code>NR</code>	Line number
<code>NF</code>	Number of fields
<code>FS</code>	Field separator
<code>OFS</code>	Output field separator

`sed` Command

`sed` stands for **Stream Editor**.

It is used for:

- Searching text
- Replacing text
- Deleting lines
- Inserting lines
- Modifying file content

It works line by line.

◆ Basic Syntax

`sed 'command' filename`

- 👉 By default, `sed` only displays output.
 - 👉 To modify the file permanently, use `-i` option.
-

📁 Example File (data.txt)

1 Ashvini 5000
2 Rahul 6000
3 Sneha 7000

◆ Common sed Operations

1 Replace Text (Substitution)

`sed 's/Rahul/Rohan/' data.txt`

- `s` → substitute
 - Replaces first match in each line
-

2 Replace Globally

`sed 's/5000/5500/g' data.txt`

- `g` → global replacement in line
-

3 Modify File Permanently

```
sed -i 's/5000/5500/g' data.txt
```

4 Delete a Line

Delete 2nd line:

```
sed '2d' data.txt
```

Delete line containing Rahul:

```
sed '/Rahul/d' data.txt
```

5 Print Specific Line

Print only 2nd line:

```
sed -n '2p' data.txt
```

- `-n` → suppress default printing
 - `p` → print
-

6 Insert Line Before

```
sed '2i New Record' data.txt
```

7 Append Line After

```
sed '2a New Record' data.txt
```

8 Replace Specific Line Number

Replace entire 2nd line:

```
sed '2c 2 Rohan 6500' data.txt
```

9 Multiple Commands

```
sed -e 's/Ashvini/Ashu/' -e 's/7000/7500/' data.txt
```

◆ Important sed Symbols

Symbol	Meaning
s	Substitute
g	Global
d	Delete
p	Print
i	Insert
a	Append
c	Change line

<code>-n</code>	Suppress output
<code>-i</code>	Edit file permanently

Difference Between `grep`, `awk`, and `sed`

Feature	<code>grep</code>	<code>sed</code>	<code>awk</code>
Full Form	Global Regular Expression Print	Stream Editor	Pattern Scanning and Processing Language
Main Purpose	Search text	Edit text	Process & analyze text
Works On	Lines	Lines	Fields (columns)
Default Output	Matching lines	Modified lines	Processed output
Supports Calculation	✗ No	✗ No	✓ Yes
Modify File	✗ No	✓ Yes (-i)	✗ Usually No
Best For	Searching	Replacing/Deleting	Reports & Data extraction

rsync – Remote Sync Command

rsync is used to **copy and synchronize files/directories** between:

- Local → Local
- Local → Remote
- Remote → Local

It is **faster than scp** because it copies only changed data (delta transfer algorithm).

◆ **Basic Syntax**

`rsync [options] source destination`

◆ **Copy Directory (Local to Local)**

`rsync -av dir1/ dir2/`

Options:

- **-a** → Archive mode (preserves permissions, ownership, timestamps)
 - **-v** → Verbose (show progress)
-

◆ **Copy File to Remote Server**

`rsync -av file.txt user@192.168.1.10:/home/user/`

👉 Copies file to remote system using SSH.

♦ **3 Copy Folder from Remote to Local**

```
rsync -av user@192.168.1.10:/home/user/data/ /backup/
```

♦ **4 Show Progress While Copying**

```
rsync -av --progress largefile.iso /backup/
```

♦ **5 Enable Compression (Faster over Network)**

```
rsync -avz folder/ user@server:/backup/
```

- **-z** → Compression
-

♦ **6 Delete Extra Files in Destination**

```
rsync -av --delete source/ destination/
```

👉 Makes destination exactly same as source.

⚠ Be careful – extra files in destination will be removed.

♦ **7 Exclude Specific Files**

```
rsync -av --exclude="*.log" source/ destination/
```

Exclude multiple:

```
rsync -av --exclude="*.log" --exclude="temp/" source/ destination/
```

♦ **8 Dry Run (Test Before Actual Execution)**

`rsync -av --dry-run source/ destination/`

👉 Shows what will happen without copying.

Very important for real admin work.

♦ **9 Preserve Specific Attributes**

`rsync -rptgo source/ destination/`

- `-r` → Recursive
- `-p` → Preserve permissions
- `-t` → Preserve time
- `-g` → Preserve group
- `-o` → Preserve owner

(`-a` includes all of these)

♦ **10 Backup with Timestamp**

`rsync -av --backup --backup-dir=/backup/old source/ destination/`

🔥 **Real-Time Admin Scenarios**

✅ **Daily Website Backup**

`rsync -avz /var/www/ user@backup-server:/backup/www/`

✅ **Sync Logs Every Hour (with cron)**

```
0 * * * * rsync -az /var/log/ user@backup:/logs/
```

✓ Mirror Production to Staging

```
rsync -av --delete prod/ stage/
```

scp – Secure Copy Command

scp (Secure Copy) is used to **transfer files securely between systems** using **SSH**.

It works over **port 22 (SSH)** and encrypts data during transfer.

◆ Basic Syntax

```
scp [options] source destination
```

◆ Copy File (Local → Remote)

```
scp file.txt user@192.168.1.10:/home/user/
```

👉 Copies **file.txt** to remote server.

◆ Copy File (Remote → Local)

```
scp user@192.168.1.10:/home/user/file.txt .
```

👉 Copies remote file to current directory.

◆ Copy Directory (Recursive)

```
scp -r folder user@192.168.1.10:/home/user/
```

- **-r** → recursive (for folders)
-

♦ **4 Specify Custom SSH Port**

If SSH runs on port 2222:

```
scp -P 2222 file.txt user@192.168.1.10:/home/user/
```

⚠ Note: **-P** is capital.

♦ **5 Enable Compression**

```
scp -C largefile.iso user@server:/backup/
```

- **-C** → compression
-

♦ **6 Preserve File Permissions & Timestamp**

```
scp -p file.txt user@server:/backup/
```

- **-p** → preserve attributes
-

♦ **7 Copy Between Two Remote Servers**

```
scp user1@server1:/data/file.txt user2@server2:/backup/
```

👉 Transfers directly between servers.

Real-Time Admin Scenarios

Send Backup File to Backup Server

```
scp backup.tar.gz user@backup-server:/backup/
```

Copy Logs from Production Server

```
scp user@prod:/var/log/app.log .
```

Upload Application Build

```
scp build.zip user@staging:/var/www/html/
```

Important Interview Points

- `scp` uses SSH protocol.
 - Requires SSH access.
 - `-r` for directory.
 - `-P` for port.
 - `-p` for preserve permissions.
 - Simpler than `rsync`.
-

Common Mistake

 Wrong:

```
scp -p 2222 file.txt user@server:/home/
```

 Correct:

```
scp -P 2222 file.txt user@server:/home/
```

Difference Between **scp** and **rsync**

Feature	scp	rsync
Encryption	✓ Yes	✓ Yes
Copy only changed data	✗ No	✓ Yes
Resume support	✗ No	✓ Yes
Best for	Simple transfer	Backup & sync
Speed for large data	Slower	Faster

alias – Create Shortcut Commands in Linux

alias is used to create **short names (shortcuts)** for long or frequently used commands.

◆ Basic Syntax

```
alias name='command'
```

◆ 1 Create Simple Alias

`alias ll='ls -la'`

Now typing:

`ll`

👉 Executes `ls -la`

◆ 2 Alias with Options

`alias grep='grep --color=auto'`

👉 Highlights matched text automatically.

◆ 3 Check All Aliases

`alias`

Check specific alias:

`alias ll`

◆ 4 Remove Alias

`unalias ll`

Remove all aliases:

`unalias -a`

♦ 5 Temporary vs Permanent Alias

♦ Temporary Alias

Created in terminal → works only in current session.

♦ Permanent Alias

For current user:

Add inside:

~/.bashrc

Example:

```
echo "alias ll='ls -la'" >> ~/.bashrc
```

```
source ~/.bashrc
```

Now it works even after logout.

System-wide:

/etc/bashrc

or

/etc/profile

♦ 6 Useful Real-Time Admin Aliases

```
alias update='sudo yum update -y'
```

```
alias ports='ss -tuln'
```

```
alias cls='clear'
```

```
alias myip='curl ifconfig.me'
```

♦ 7 Override Default Command

```
alias rm='rm -i'
```

👉 Asks confirmation before deleting.

⚠️ Useful for safety in production.

14 **ssh** – Secure Shell (Detailed Explanation)

♦ What is SSH?

ssh (Secure Shell) is a **network protocol + command-line tool** used to:

- Connect to remote servers securely
- Execute commands remotely
- Transfer files (via scp/sftp)
- Perform remote administration

It encrypts communication between client and server.

Default Port: **22**

Why SSH is Important?

Before SSH → Telnet was used ❌ (Not encrypted)

SSH provides:

-  Encryption

-  Authentication
-  Integrity protection

So passwords & data cannot be sniffed on network.

How SSH Works (Step-by-Step)

When you run:

```
ssh user@server_ip
```

Step 1 – Connection Establishment

Client connects to server on port 22.

Step 2 – Key Exchange

Client and server exchange cryptographic keys.

Step 3 – Authentication

User is authenticated using:

- Password authentication
- Public/Private key authentication

Step 4 – Encrypted Session Starts

Now all communication is encrypted.

SSH Authentication Methods

1 Password Authentication

ssh user@192.168.1.10

Server asks password.

Simple but less secure.

Key-Based Authentication (Recommended)

Generate Key:

ssh-keygen

Creates:

~/.ssh/id_rsa (Private key)

~/.ssh/id_rsa.pub (Public key)

Copy Public Key to Server:

ssh-copy-id user@server

Now login without password:

ssh user@server

More secure & used in automation.

Important SSH Files

File	Purpose
~/.ssh/id_rsa	Private key (Never share)

`~/.ssh/id_rsa.pub` Public key

`~/.ssh/authorized_keys` Allowed keys

`/etc/ssh/sshd_config` SSH server configuration

Important SSH Options

Connect using key:

```
ssh -i mykey.pem user@server
```

Use custom port:

```
ssh -p 2222 user@server
```

Verbose mode (Troubleshooting):

```
ssh -v user@server
```

SSH Port Forwarding (Advanced Concept)

Used for accessing remote services securely.

Local Port Forwarding

```
ssh -L 8080:localhost:80 user@server
```

Access remote port 80 via:

`http://localhost:8080`

Remote Port Forwarding

`ssh -R 9090:localhost:3000 user@server`

SSH Hardening (Security Best Practices)

Edit:

`sudo vi /etc/ssh/sshd_config`

Common security settings:

`PermitRootLogin no`

`PasswordAuthentication no`

`Port 2222`

Restart service:

`sudo systemctl restart sshd`

SSH Service Management

Check status:

`systemctl status sshd`

Start:

`sudo systemctl start sshd`

Enable at boot:

```
sudo systemctl enable sshd
```

Common SSH Errors & Fix

1 Permission denied (publickey)

Fix key permission:

```
chmod 600 mykey.pem
```

2 Connection refused

- SSH service not running
 - Firewall blocking port
 - Wrong port
 - Cloud security group blocked
-